

Cornerstone LUC — Scaling & Reliability Assessment

What a live cloud run tells us about building on this pipeline

September 2026

1. Executive summary

We ran the land-use-change (LUC) attribution pipeline end-to-end on Google Kubernetes Engine (GKE) — all four phases, on real country data — to learn how it behaves under cloud execution and where it would strain if we scaled it toward continental or global coverage. This note is written for that decision: **it emphasises the failure modes we did *not* hit as much as the ones we did**, because de-risking is as much about knowing what is already solid as about knowing what needs work.

The headline: the architecture is sound and cheap to run at its current 30 m resolution. A whole-world (land) run is **279 tiles**, costs **tens of dollars** and **~110 pod-hours to compute**, and the failure we did hit (disk I/O) was diagnosed and fixed, turning a 45-minute stall into a 9.5-minute run. The retry-safe caching means a re-run of finished work is a near-instant no-op (measured: **39 s vs 9.5 min**).

Where the risk actually lives is not compute and not reliability — it is **storage cost, and how it scales with resolution**. Compute is one-time and cheap; stored data is recurring and grows $\sim 9\times$ when we move from 30 m to 10 m. That, plus the fact that our largest-scale test so far is only two tiles, defines the work that remains before large-scale production.

2. Scope of the evidence (read this before the numbers)

Everything below extrapolates from a **live 2-tile run over Czechia (CZE)**, plus an earlier Honduras run. This is a **functional pilot, not a scale test**. It is enough to characterise per-tile behaviour, cost structure, and the qualitative bottlenecks with confidence; it is **not** enough to prove behaviour at hundreds of concurrent pods. Per-tile compute time in particular can plausibly swing $\pm 2\times$ across the globe (tropical farmland tiles are heavier than European ones), so whole-world compute figures are order-of-magnitude, not $\pm 20\%$. We flag this throughout rather than hide it — the honest scope *is* part of the de-risking.

3. The architecture, in brief

The pipeline runs as four sequential phases, each a Kubernetes *Indexed Job*:

ingest-world $\rightarrow$ ingest-tiles $\rightarrow$ compute $\rightarrow$ reduce

- **One pod per tile.** Each pod maps its job-completion index onto the sorted list of tiles the target countries touch. Fan-out is a single parallelism knob against the cluster autoscaler.

- **Scale-to-zero.** Nodes spin up on demand and drain to nothing when idle, so the cluster costs nothing between runs.
- **Cache = automatic retry.** Every stage is wrapped in a content-addressed cache, so a retry is just a re-run and any already-finished tile is a no-op.

Why this shape de-risks: it is deliberately simple — no bespoke workflow engine, no long-lived cluster, no hand-managed state. Fewer moving parts means fewer things that can break, and “just run it again” is always a safe recovery.

4. Risks we did not hit

This is the core of the de-risking case. Each item is backed by a measurement from the run.

Risk	What we observed
Out-of-memory crashes	Compute peaked at 85% of its ceiling and never OOM'd; memory sizing is correct, not lucky.
Disk I/O throttling	After our fix, I/O pressure fell to ~0; local scratch used <1% on compute.
Scheduling deadlock / stuck pods	Pods scheduled promptly and drained cleanly; the earlier stall was the disk issue, now resolved.
Cache corruption / silent recompute	Idempotency held; a warm re-run did zero work (39 s). Correctness verified by per-band checksums.
Autoscaler failure	Nodes came up on demand, packed correctly, and scaled back to zero.
Credential / security friction	Workload Identity worked; the source-API key is confined to the one phase that needs it.
Cross-region egress surprise	The bulk dataset streamed from in-region storage at full speed (~140 MB/s).

None of these required special handling. That is the most reassuring part of the result: the common ways a cloud batch pipeline fails simply did not occur.

5. The issue we did hit — and fixed

The one real problem was **disk I/O throttling during ingest**. The GLAD land-cover step wrote an 8 GB uncompressed intermediate per tile, which saturated local disk and stalled the pod — the first 2-tile attempt hung for ~45 minutes.

The fix was a **one-parameter change** to how that intermediate is written (band-interleaved + light compression): **8.0 GB → 1.15 GB (~7× smaller), and faster to write**, with output proven bit-identical by checksum. Combined with moving scratch onto a dedicated per-pod SSD, the same run then completed in **9.5 minutes** with I/O pressure near zero.

The point for stakeholders is not the megabytes — it is that an infrastructure problem surfaced, was root-caused, and was fixed with a surgical change and no architectural disruption. That capability is itself a de-risking asset.

6. Bottlenecks & scaling behaviour, by phase

- **Ingest** — CPU-bound on format conversion once the disk issue was removed. Network is a *secondary* cost, and only for a few externally-hosted datasets (third-party academic servers), not

the in-region bulk data.

- **Compute** — the long pole, ~17–20 min/tile. **Memory-bound** (peaks ~51 GiB of a 64 GiB node), so one pod per node is correct. The dominant phase is a **single-threaded loop over provinces × crops** using ~1 of 8 cores — so we pay for an 8-core node and use one during the part that takes longest. Not disk-bound; scratch is barely touched.
- **Reduce** — trivial (seconds). It is fast because compute already produced and cached every per-tile partial; reduce just sums them. This speed is a property of the cache and phase split, **not** of the methodology.

Scaling character: the pipeline scales *out* (across tiles) beautifully — tiles are independent. It scales *up* (within a tile) poorly, because the long pole is serial and memory is already near the node ceiling.

7. Cost model (public us-central1 pricing)

Rates used (on-demand, current public pricing): **e2-highmem-8 = \$0.362/node-hour**; **pd-ssd = \$0.17/GB-month**; **Cloud Storage Standard = \$0.020/GB-month**. Spot instances are ~one-third of on-demand.

Per tile

Item	30 m
Compute (one-time)	~\$0.11–0.13
Ingest, cold (one-time)	~\$0.03
Reduce	~\$0.00
Full process, on-demand	~\$0.13–0.16 / tile
Full process, Spot	~\$0.05–0.08 / tile
Durable storage (recurring)	4.5 GB → \$0.09 / tile-month

Per-tile durable storage is **2.47 GB** of ingest rasters + **2.0 GB** of emissions layer.

Whole world (land) — 279 tiles, 220 countries

30 m	10 m (~9× pixels)
Compute ~110 pod-hours	~780 (same cheap nodes)
Compute ~\$35–40 cost, on-demand (one-time)	~\$320–360
Compute ~\$13–15 cost, Spot	~\$110–130

	30 m	10 m ($\sim 9\times$ pixels)
Durable storage (recurring)	~ 1.25 TB $\rightarrow$ $\sim \$25/\text{mo}$ ($\sim \$300/\text{yr}$)	~ 11 TB $\rightarrow$ $\sim \$225/\text{mo}$ ($\sim \$2,700/\text{yr}$)

The shape of the business case: at 30 m, computing the whole world is *trivially cheap and one-time*; the recurring **storage** bill overtakes it within ~ 2 months and continues forever. **Storage — not compute — is the cost to manage**, and it is the one that scales badly with resolution.

The 10 m jump — cheap-node, not big-node

Moving 30 m $\rightarrow$ 10 m is $3\times$ linear = $\sim 9\times$ **the pixels**, and the important finding is that this **does not inflate memory**. The pipeline chunks work to a constant **~ 1 GiB per chunk in bytes** (chunk pixel-dimensions shrink automatically at higher resolution), and Dask runs as one worker with a configurable thread count (currently 8). Peak memory is therefore **threads $\times$ ~ 1 GiB + overhead — independent of resolution**: the 30 m tile peaked ~ 51 GiB at 8 threads, and a 10 m tile peaks the *same* ~ 51 GiB at 8 threads (there are simply $9\times$ more chunks to stream). With **1 thread it drops to ~ 10 – 15 GiB**.

Consequently **10 m needs no bigger node and no re-architecture**. It runs on the same e2-highmem-8 (or e2-highmem-16 for headroom, or 1-worker on a small node), and the $9\times$ shows up as **wall-clock time**, so whole-world 10 m compute is **$\sim \$320$ – 360 on-demand** — not thousands. The genuine cost jump at 10 m is **storage** ($\sim 9\times$, recurring), not compute.

(One thing to verify empirically before committing to 10 m: that the per-province clip/populate step also stays chunk-bounded rather than materialising a whole province at once — at 30 m that phase used only ~ 3 – 4 GiB, so even a $9\times$ worst case is well within a cheap node.)

8. Scaling levers from here

Lever	Verdict
Scale up (bigger pod, more in-pod concurrency)	Limited. Memory is already $\sim 85\%$ per tile, so more concurrency spills to local disk and reintroduces the ingest disk-I/O failure — and the serial long pole doesn't parallelise anyway.
Scale out (more tiles in parallel)	The proven, low-risk lever. Splits trivially per tile; cache makes retries free. New limits at large scale are cloud quotas, source-API rate limits, and autoscaler latency — not the pipeline itself.
Tweak CPU / memory / disk	Marginal, linear. Memory can't go down (pinned by the 85% peak); disk is over-provisioned and could be trimmed. Won't move bottlenecks.
Spot instances	Strong cost lever ($\sim$ two-thirds off). Correctness risk is low — the workload is idempotent and cache-safe. Costs: no mid-tile checkpoint (a preemption wastes up to ~ 20 min of a tile), possible capacity scarcity for large nodes, minor retry dev.

The one genuinely **unexplored** seam is **per-country aggregation**: reduce runs as a *single pod per area of interest*. It was trivial here (2 tiles), but a many-tile country or continent funnels all partials through one pod, and we have not measured whether that pod slows down at breadth.

9. Where we'd hit constraints if we build on this

Risk	Likelihood	Impact	Mitigation	Effort
Scale unproven beyond 2 tiles (quotas, API rate limits, autoscaler latency)	High at global scale	Medium	A staged scale test with pre-provisioned quota	Moderate
Storage cost growth , esp. at 10 m	High	High (recurring)	Lifecycle/tiering (Nearline/Coldline), compression, keep only what's needed	Low-moderate
External data-source dependencies (third-party hosts, no SLA)	Medium	Medium	Mirror sources into our own storage	Low
Compute long pole is single-threaded	Certain	Medium (per-tile latency & 10 m cost)	Parallelise the province loop, or sub-tile chunking	Moderate
Per-tile memory variance (Euro ~51 GiB vs heavier tiles)	Medium	Medium (OOM on heavy tiles)	Profile heavy tiles; size or split accordingly	Low
Driver is a single polling process	Low	Low (cache makes resume safe)	Leave as-is until scale demands otherwise	—
Observability is homegrown (stdout logs)	Certain	Low-medium in production	Aggregate metrics + basic alerting	Moderate

10. Solidity assessment & roadmap

Solid today — safe to build on as-is:

- Idempotent cache / retry semantics (proven: 39 s warm re-run).
- Phase isolation and scale-to-zero economics.
- The disk-I/O fix.
- Security posture (Workload Identity, scoped credentials).
- Verified numerical correctness (checksums).

Solid with moderate work:

- **Storage lifecycle & tiering** — the highest-value item, because storage is the dominant recurring cost. Move cold data to cheaper classes; decide retention.
- **Mirror external data sources** into our storage to remove third-party reliability risk.
- **A staged scale test** (tens → hundreds of tiles) with pre-provisioned quota, to convert “unproven” into “proven”.
- **Basic observability** — aggregate the metrics we already emit, add alerting.
- **Parallelise the compute long pole** (or sub-tile chunking) — becomes important specifically for 10 m.

Needs a decision only at large scale: whether the single-process driver and single-pod reduce warrant hardening. Neither is a problem today.

Bottom line: nothing we saw calls the architecture into question. The pipeline is cheap and reliable to run at 30 m; the real forward work is **managing storage cost and proving scale**, both of which are moderate, well-understood efforts rather than open-ended risks.

Appendix A — Measured run data

Run: AOI Czechia (CZE), 2 tiles (50N_010E, 60N_010E), all four phases, GKE, us-central1, e2-highmem-8 nodes.

Phase	Duration	Key resource facts
ingest-tiles (cold, prior run)	9 m 30 s	was 45-min stall before disk fix
ingest-tiles (warm re-run)	39 s (3.1 s work/tile)	zero downloads/conversion — cache hit
ingest-world (warm)	31 s	one-time per run
compute — tile 50N_010E	1224.6 s (20.4 m)	peak mem 84.9%; CPU throttled 0 s; localtmp 0.2%
compute — tile 60N_010E	1015.9 s (16.9 m)	peak mem 85.6%; I/O pressure ≤ 2.76 ; write 0.09 GiB
reduce (CZE)	34 s (11.3 s work)	peak mem 0.5%

Disk fix (GLAD intermediate): 8.0 GB $\rightarrow$ 1.15 GB ($\sim 7\times$), band-interleave + ZSTD-1, output checksum-identical; run 45 min $\rightarrow$ 9.5 min.

Data sizes (per tile): ingest COGs 2.47 GB (GLAD 1.50, Harris-AGB 0.96, soilgrids/huang/peatlands ~ 0.02); emissions-layer zarr 2.00 GB; attribution parquets ~ 0.02 – 0.03 MB each.

Whole world: 279 land tiles across 220 countries (computed from the pipeline’s own tiling).

Appendix B — Pricing sources

- e2-highmem-8: **\$0.362/hour** on-demand, us-central1 (8 vCPU / 64 GB) — consistent with component pricing $8 \times \$0.0218 + 64 \times \0.0029 .
- n2-highmem-64: **\$4.19/hour** (64 vCPU / 512 GB); n2-highmem-80: \$5.24/hour.
- Persistent Disk SSD (pd-ssd): **\$0.17/GB-month**; pd-standard: \$0.04/GB-month.
- Cloud Storage Standard, regional: **\$0.020/GB-month**.
- Spot instances: roughly one-third of on-demand.

All figures us-central1, current public list pricing as of September 2026; $\pm 20\%$ is acceptable for the decisions this note supports.